

实验十 BP 神经网络

（一）目的和要求

- （1）了解 BP 神经网络的基本原理和结构。
- （2）掌握 BP 神经网络的前向传播和反向传播算法。
- （3）理解 BP 神经网络在分类问题中的应用。
- （4）了解 BP 神经网络与其他神经网络的区别。

（二）实验知识准备

（1）什么是 BP 神经网络

BP 神经网络（Backpropagation Neural Network）是一种**多层前馈神经网络**，通过**反向传播算法**进行训练。以下是 BP 神经网络的主要特点和组成部分：

1. **前向传播**：输入数据通过网络层，每层的神经元对输入进行加权求和，并通过激活函数进行非线性变换。
2. **反向传播**：计算输出层的误差，并将误差反向传播到网络的每一层，更新每层的权重。
3. **激活函数**：如 Sigmoid、Tanh、ReLU 等，用于引入非线性，使网络能够学习复杂的模式。

（2）BP 神经网络与其他神经网络的区别

BP 神经网络主要通过反向传播算法进行训练，而**其他神经网络如卷积神经网络（CNN）和循环神经网络（RNN）有其特定的结构和训练方法**。

BP 神经网络适用于较小规模的数据集和简单的任务，而 CNN 和 RNN 在处理图像和序列数据时更为有效。

（3）BP 神经网络结构

BP 神经网络一般由输入层、隐藏层及输出层组成，如图 3 所示。

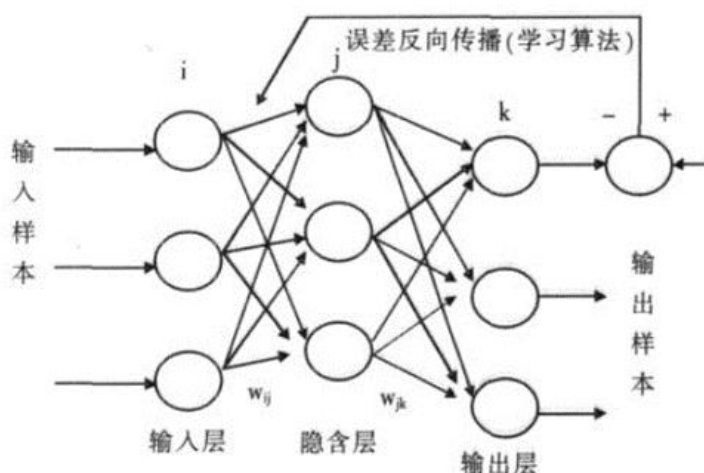


图 3 BP 神经网络结构

其中，网络结构主要由以下几个部分说明：

1. 神经网络结构：

- (1) 输入层：神经网络的第一层，接收输入样本。
- (2) 隐含层（或隐藏层）：位于输入层和输出层之间的层，负责提取和组合输入数据的特征。
- (3) 输出层：神经网络的最后一层，产生最终的输出结果，如分类标签。

2. 权重 (W)：

W_{jk} ：表示连接隐含层的第 j 个神经元和输出层的第 k 个神经元之间的权重。权重是神经网络中的关键参数，它们决定了输入信号如何影响输出。

3. 误差反向传播过程：

- (1) 在训练过程中，神经网络会接收输入样本，并产生输出样本。
- (2) 这些输出样本与实际的目标输出样本（标签）进行比较，计算出误差。
- (3) 误差反向传播算法的核心是将这个误差从输出层反向传播回输入层，通过调整每一层之间的权重来最小化误差。

4. 学习算法：

误差反向传播算法是一种监督学习算法，它通过梯度下降方法来优化权重，以减少预测误差。

在每次迭代中，算法计算损失函数（如均方误差）关于每个权重的梯度，然后更新权重以减少损失。

5. 权重更新:

权重的更新是通过学习率(通常表示为 η)和梯度的乘积来实现的,即 $W = W - \eta * \partial E / \partial W$, 其中 E 是损失函数, $\partial E / \partial W$ 是损失函数关于权重的偏导数。

(4) 学习过程

1. 正向传播

输入信号从输入层经过各个隐含层向输出层传播,在输出层得到实际的响应值,若实际值与期望值误差较大,就会转入误差反向传播阶段。

2. 反向传播

按照梯度下降的方法从输出层经过各个隐含层并逐层不断地调整各神经元的连接权值和阈值,反复迭代,直到网络输出的误差减少到可以接受的程度,或者进行到预先设定的学习次数。

BP 神经网络通过有指导的学习方式进行训练和学习。标准的 BP 算法采用误差函数按梯度下降的方法学习,使网络的设计输出值和期望输出值之间的均方误差最小。BP 神经网络的传输函数通常采用 sigmoid 函数,而输出层则采用线性传输函数。

（三）实验内容及步骤

（1）绘制函数的 3D 图像，示例代码如图 1 所示。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 定义函数
5 def fun(x, y):
6     return -(np.power(x, 2) + np.power(y, 2))
7
8 # 创建 3D 图形对象
9 fig = plt.figure()
10 ax = fig.add_subplot(111, projection='3d') # 添加3D子图
11
12 # 生成数据
13 X0 = np.arange(-3, 3, 0.25)
14 X1 = np.arange(-3, 3, 0.25)
15 X0, X1 = np.meshgrid(X0, X1)
16 Z = fun(X0, X1)
17
18 # 绘制曲面图，并使用 cmap 着色
19 surf = ax.plot_surface(X0, X1, Z, cmap=plt.cm.winter)
20 ax.set_xlabel('X0', color='r')
21 ax.set_ylabel('X1', color='g')
22 ax.set_zlabel('f(x)', color='b') # 给三个坐标轴注明坐标名称
23
24 # 添加颜色条
25 fig.colorbar(surf)
26
27 plt.show()
```

图 1 绘制函数图像

图 1 所示代码中，利用 Python 编程语言中的 matplotlib 和 numpy 库，首先定义了一个数学函数 fun，该函数计算给定 x 和 y 值的负平方和。接着，代码创建了一个 3D 绘图环境，生成了一个从-3 到 3 的 x 和 y 值的网格，计算了每个点的函数值，并将这些值作为 z 轴的高度。然后，代码使用`plot\_surface`方法在 3D 空间中绘制了一个曲面图，其中颜色映射（cmap）用于根据函数值的不同显示不同的颜色，最后设置了坐标轴的标签并显示了这个 3D 图形。这个过程直观地展示了一个向下凹陷的抛物面形状，其顶点位于原点，且随着 x 和 y 值的增加，函数值（z 轴高度）减小。结果如图 2 所示

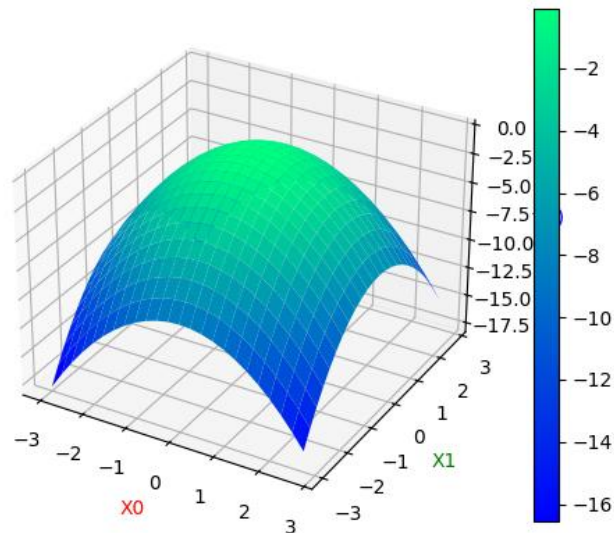


图 2 函数结果

图 2 可看出，当 $x_0=0$ 且 $x_1=0$ 时，函数达到最大值。意味着函数达到最大值处，极值为 0。

(2) 求函数的梯度。求函数 $y = x^2 + 2x - 1$ 在 $x=2$ 处的梯度，示例代码如图 3 所示。

```
1 import tensorflow as tf
2
3 def func(x):
4     return x ** 2 + 2 * x - 1    #1元函数
5
6 def gradient_test():
7     x = tf.constant(value = 2.0)
8     with tf.GradientTape() as tape:
9         tape.watch(x)
10        y = func(x)
11    #一阶导数
12    order_1 = tape.gradient(target = y, sources = x)
13    print("函数  $x^2 + 2x - 1$  在  $x = 2$  处的梯度为: ", order_1.numpy())
```

图 3 求函数的梯度

图 3 所示代码中，TensorFlow 框架中的 GradientTape 上下文管理器来计算函数 $y = x^2 + 2x - 1$ 在 $x = 2$ 处的一阶导数。GradientTape 自动记录在代码块中执行的运算，以便之后可以计算梯度。在这个代码块中， x 被设置为常量 2.0，并被 `tape.watch` 监控，此时 x 的变化将被跟踪。然后计算 $y = \text{func}(x)$ 的值。一旦退出 `with` 代码块，`tape.gradient` 方法被调用来计算 y 关于 x 的梯度，即函数在 $x = 2$ 处的一阶导数，并打印结果。

(3) 求函数的二阶梯度。求三元函数 $y = x^2 + 2x - 1$ ，在 $x=2$ 处的二阶梯度。示例代码如图 4。

```

1 import tensorflow as tf
2
3 x = tf.Variable(2.0, trainable=True)
4 with tf.GradientTape() as tape1:
5     with tf.GradientTape() as tape2:
6         y = x ** 2 + 2 * x - 1
7         order_1 = tape2.gradient(y, x)
8     order_2 = tape1.gradient(order_1, x)
9
10 print("在x = 2处的一阶梯度为:", order_1.numpy())
11 print("在x = 2处的二阶梯度为:", order_2.numpy())

```

图 4 求函数的二阶梯度

图 4 所示代码使用了 TensorFlow 中的 GradientTape 来计算函数 $y = x^2 + 2x - 1$ 在 $x = 2$ 处的一阶和二阶导数。首先，创建了一个可训练的 `tf.Variable` 对象 `x` 并初始化为 2.0。然后，使用两层嵌套的 GradientTape 上下文管理器：内层 `tape2` 用于计算 y 关于 x 的一阶导数，即函数值的变化率；外层 `tape1` 用于计算一阶导数 `order_1` 关于 x 的导数，即二阶导数。通过这种方式，代码首先计算出在 $x = 2$ 时函数的一阶导数，然后进一步计算这个一阶导数自身的导数，也就是二阶导数，并分别打印出这两个导数值。

(4) 求三元函数的梯度。求三元函数 $y = x^3 + 3xy + y^2 + z^3$ ，在 $x=1$ ， $y=2$ 和 $z=3$ 处的梯度。示例代码如图 5。

```

1 def func(x): # 注意：此处x为一个多元张量
2     return x[0] ** 2 + 3 * x[0] * x[1] + x[1] ** 2 + x[2] ** 3
3
4 def gradient_test(): # 求n(=3)元函数的梯度
5     x = tf.Variable([1.0, 2.0, 3.0], dtype=tf.float32) # 使用tf.Variable而不是tf.constant
6     with tf.GradientTape() as tape:
7         tape.watch(x)
8         y = func(x)
9     # 一阶导数
10    order_1 = tape.gradient(target=y, sources=x)
11    # 因为x是一个张量列表，所以order_1也是一个张量列表
12    print("多元函数x = [1.0, 2.0, 3.0]处的梯度为:", [grad.numpy() for grad in order_1])
13
14 if __name__ == "__main__":
15     gradient_test()

```

图 5 三元函数的梯度

图 5 所示代码定义了一个多元函数 `func`，它接受一个包含三个元素的张量 `x`，并计算一个关于 `x` 的多项式函数值。在 `gradient_test` 函数中，首先创建了一个 `tf.Variable` 对象 `x`，其初始值为 `[1.0, 2.0, 3.0]`，并指定数据类型为 `tf.float32`。然后，使用 `tf.GradientTape` 上下文管理器来记录 `func` 函数的前向计算过程，并监控 `x`

的变化。计算得到函数值 y 后，通过 `tape.gradient` 方法计算 y 关于 x 的梯度，即一阶导数。由于 x 是一个包含三个元素的张量，所以得到的梯度 `order_1` 也是一个包含三个梯度值的列表。最后，代码使用列表推导式将每个梯度值转换为 NumPy 数组，并打印出来，展示了在点 $[1.0, 2.0, 3.0]$ 处，函数对每个变量的敏感度或变化率。

(5) 基于梯度递减的线性回归，示例代码如图 6。

```
1 import numpy as np
2 import random
3 import tensorflow as tf
4
5 # 定义损失函数
6 def loss(y_true, y_pred):
7     return tf.keras.losses.MeanSquaredError()(y_true, y_pred)
8
9 # 生成训练数据
10 x_train_inch = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11], dtype=np.float32)
11 y_train_price = np.array([num * 10 + 5 for num in x_train_inch], dtype=np.float32) # y = 10x + 5
12
13 # 随机初始化待训练的参数
14 w1 = tf.Variable(random.random(), trainable=True, dtype=tf.float32)
15 w0 = tf.Variable(random.random(), trainable=True, dtype=tf.float32)
16
17 def step_gradient(real_x, real_y, learning_rate):
18     with tf.GradientTape() as tape:
19         # 计算模型预测
20         pred_y = w1 * real_x + w0
21         # 计算损失
22         reg_loss = loss(real_y, pred_y)
23         # 计算梯度
24         w1_gradients, w0_gradients = tape.gradient(reg_loss, (w1, w0))
25         # 更新权值
26         w1.assign_sub(w1_gradients * learning_rate)
27         w0.assign_sub(w0_gradients * learning_rate)
28
29 if __name__ == '__main__':
30     learning_rate = 0.01
31     num_iterations = 1000
32
33     for _ in range(num_iterations):
34         step_gradient(tf.convert_to_tensor(x_train_inch), tf.convert_to_tensor(y_train_price), learning_rate)
35
36     print(f'拟合得到的模型近似为:  $y \approx \{w1.numpy()\}x + \{w0.numpy()\}$ ')
37     # 预测
38     wheat = 0.9
39     price = w1 * wheat + w0
40     print("price = {0:.2f}".format(price.numpy()))
```

图 6 线性回归

图 6 所示代码使用 TensorFlow 框架实现。首先定义了一个均方误差损失函数，然后生成了一组训练数据，其中 `x_train_inch` 是输入特征，`y_train_price` 是对应的目标值，两者的关系为 $y = 10x + 5$ 。接着，代码初始化了两个模型参数 `w1`（斜率）和 `w0`（截距），并在主函数中通过梯度下降法迭代更新这些参数，以

最小化损失函数。每次迭代中，使用 `tf.GradientTape` 来记录和计算损失函数关于参数的梯度，然后根据这些梯度和学习率更新参数。最终，打印出拟合得到的线性模型参数，并使用这些参数预测给定输入 `wheat = 0.9` 时的价格。

(6) 实训项目

实现了一个简单自定义的 **BP** 神经网络，用于对鸢尾花数据集进行分类。该程序的应包括自定义方法计算 BP 神经网络的前向传播与反向传播的过程。